

EsCapturer: Detecting Cgroups Escape Attacks For Container Clouds Through Behavior-Intention Learning

Dongyang Zhan, *Member, IEEE*, Kai Tan*, Xu Liu, Haining Yu, Bei Zhao, and Zhihong Tian

Abstract—The proper execution of containers is very important for cloud native, but the Linux cgroups mechanism at the bottom of containers is vulnerable, which can be compromised by attackers to interfere with normal cloud native services. Since cgroups is an underlying mechanism of Linux, it is difficult to change it to solve the problem. Anomaly behavior detection is widely used in container security, but the existing anomaly detection methods do not perform well in solving this problem because the behaviors of this kind of attack are similar to normal behaviors. In this paper, a behavior analysis approach based on behavior-intention learning, EsCapturer, is proposed to detect the cgroups escape attacks in containers. Based on our observation, behavior intentions (e.g., behavior target, operations, etc.) are the key factors in identifying anomaly behaviors. To this end, a multilevel deep learning model is proposed to learn the behavior intention features of local related behaviors and the contextual features among different behavior intentions, which can detect attacks precisely. The evaluation results have shown that EsCapturer can well address the problem of cgroups escape detection and has outperformed other existing log-based anomaly detection methods.

Index Terms—Cgroups escape detection, behavior intention analysis, container security, cloud native.

I. INTRODUCTION

THE container technology is considered to be the standard for cloud native, and the representative technology is Docker containers[1]. Compared with virtual machines, Docker containers share the host kernel, making them more lightweight and efficient. But sharing the host kernel makes the isolation of Docker containers weaker than that of virtual machines. Some security concerns about container security have been raised, such as resource isolation[2], privilege escalation[3], information leaks[4], etc.

There are two key components in the Linux kernel to isolate different containers, including Linux Namespaces (i.e., namespaces) and Linux Control Groups (i.e., cgroups). The namespaces and cgroups are used for isolating the resource view and the resource consumption, respectively. The cgroups mechanism is a Linux kernel feature that is implemented in the core kernel to allow processes to be organized into hierarchical groups. The resource usage of each group, including CPU,

block I/O, memory, et al., can be monitored and limited. Based on the cgroups mechanism, processes in each container can be limited to consuming the system resources of the host. Therefore, resource consumption attacks from containers can be defended against by applying the cgroups policies[3].

However, the cgroups mechanism was found to be vulnerable in the container and cloud native scenarios[2]. Processes running inside Docker containers can generate workloads that are not managed by the container cgroups, so the workloads will not be charged to the container cgroups. By using this approach, containers can consume more resources on the host or perform a denial-of-service attack, which is very dangerous for cloud computing.

It is not easy to mitigate the cgroups escape attacks because they are caused by the underlying kernel mechanisms (e.g., kernel threads, interrupts, and system services). The cgroups mechanism is performed by attaching resource management controllers to system resources, ensuring that the processes belonging to one cgroup (e.g., processes of one container) will not consume extra host resources. However, some underlying kernel mechanisms are not considered in the implementation of cgroups. To address these problems, the corresponding underlying mechanisms should be redesigned, which is not practical for commercial cloud computing due to its complexity and reliability. Furthermore, the complex and diverse application environments in the cloud, with various unknown applications running simultaneously and containing a wide range of normal behavior patterns, make behavior analysis particularly important in this highly dynamic setting. Fortunately, detection methods based on the analysis of attack behavior patterns can identify such threats, which is a widely used technology in software engineering[5].

There are several challenges in detecting the attack behaviors in containers. First, the overhead introduced by the behavior interceptor should be very low, and many kinds of execution events should be monitored. But the current behavior monitors[6] for containers can only intercept syscall execution, which cannot capture all of the attack behaviors, such as kernel upcalls, etc. Second, the attack behaviors are similar to normal behaviors, so it is not easy to model the attack behaviors.

The existing learning-based anomaly detection approaches do not perform well in container security analysis[7]. That is because they mainly focus on the behavior sequence or frequency, which is not enough for container attack scenarios. In this scenario, the same operations with different targets and arguments can cause different effects. For instance, writing to

D. Zhan, K. Tan, L. Ye, X. Liu are with the School of Cyberspace Science, Harbin Institute of Technology, Harbin, Heilongjiang, 150001.

B. Zhao is with the China Mobile Communications Group Co., Ltd.

Z. Tian is with the Guangzhou University, Guangzhou, 510000.

* Corresponding Author: tankai@hit.edu.cn

Manuscript received ; revised .

system logs can introduce extra workload to system services, which can be used to perform the resource consumption attack, but writing to a regular file does not. Therefore, more semantic information and behavior features should be analyzed in container anomaly detection.

In this paper, we propose EsCapturer, a novel deep-learning-based cgroups escape attack detection approach that can intercept and analyze container behaviors accurately with low overhead. To capture the attack-related container behaviors, the attack vectors are first analyzed. We make a trade-off between the introduced overhead and detection capability, so only the necessary events in the kernel are intercepted, including some kernel functions (e.g., the interfaces of upcalls and kernel thread management) and system calls. Based on our observation, the execution of a program consists of a collection of behavior sets, and each set of behaviors has its own behavior intention. For example, reading a file and sending it to other devices in the network. Among other things, file open, read, and close can comprise a set of file-reading behaviors that have the intention of reading the content of a specific size of a regular or log file. The combination of multiple behavior intentions constitutes a complete program functional goal, and the behavior intention feature can describe the goal and comprehensive semantics of behaviors, which can be used to distinguish between attack behaviors and normal behaviors.

To detect cgroups escape attacks through behavior intention learning, a behavior correlation analysis approach is proposed to correlate behaviors that have the same intention systematically. Then, a multilevel deep learning model is proposed to learn the behavior intention of each behavior set and the contextual intention feature among sequential behavior sets, which can detect attacks precisely. To the best of our knowledge, EsCapturer is the first work to analyze and learn the behavior intention feature in deep-learning-based container anomaly detection.

In summary, the contributions of our paper are as follows.

- Based on the analysis of the attack vectors of the cgroups escape attacks, a host-based fine-grained low-overhead container behavior monitor is proposed to capture the attack-related behaviors.
- A deep-learning-based cgroups escape attack detection approach, EsCapturer, is proposed by learning and analyzing the behavior intention features of local and global behavior sets, which can represent the behavior semantics comprehensively.
- We have evaluated EsCapturer on the effectiveness and performance aspects. The results show that our approach can accurately detect the cgroups escape attacks with negligible overhead and has outperformed other existing dynamic anomaly detection approaches. The code and dataset will be released after the acceptance.

The remainder of this paper is organized as follows. Section II gives the motivation. The design of EsCapturer and the preparation of data are described in Section III. Section IV presents the anomaly detection model based on behavior-intention learning. Section V describes some important details in implementation. Section VI evaluates the effectiveness

and performance of the prototype system. The discussion is presented in Section VII. The related work is summarized in Section VIII. Section IX concludes this paper.

II. MOTIVATION

Docker containers are widely applied in cloud computing environments, which package up software and all its dependencies in standard units so that they can run in different environments. Unlike virtual machines, Docker containers share the host's operating system kernel, so they are more lightweight and efficient. With containers, microservice[8] and Function as a Service (FaaS)[9] computing architectures are proposed, which are two popular architectures in cloud native.

Since Docker containers share the host kernel, there are two main isolation mechanisms to isolate different containers on one host, which are cgroups and namespaces. The namespaces are designed to isolate different kinds of resources, such as files, processes, networks, etc. Cgroups (abbreviated from control groups) limit the resource usage (e.g., CPU, memory, network bandwidth, etc.) of a process. There are many resource controllers provided by the Linux kernel for the cgroups. The controllers can manage the resources assigned to processes. A cgroup can consist of many processes that are bound to the same resource limits. From the perspective of processes with the same cgroups, they are running on an independent machine. So, one of its design goals is to provide operating-system-level virtualization, such as containers. Cgroups can prevent processes in one container from draining all the resources of the host.

A. Cgroups Escape Attacks

Due to the complexity of the Linux kernel, some mechanisms are not considered in the cgroups implementation, raising the risk of cgroups escape attacks. The cgroups escape attacks are first discussed in [2], which explores the possible ways to escape from the container cgroups. The core idea is to inject workloads into system components that are not managed by the container cgroups. The injected workloads will consume the system resources and perform the DoS attack on the cloud host.

There are several mechanisms can be leveraged by a process to inject workloads into other system components.

1) Kernel threads. Kernel threads are created by the Linux kernel, so they belong to the root cgroup. In addition, the processes created by kernel threads are also attached to the root cgroup. If a container process can generate workloads into kernel threads, it can break the limitations of the container cgroups.

To perform the attack, a container process can let a kernel thread create a new process (e.g., triggering a kernel upcall). The workloads of the newly created process will not be charged to the container cgroup, because the process is attached to the root cgroup. Therefore, the attackers in a container need to trigger kernel threads to let the threads upcall other user-space processes. The write-back mechanism for disk data synchronization can also be used to perform the attack. When writing data to the disk, the operating system

will only write it to the cache instead of the disk device to optimize performance. However, if an attacker makes the kernel synchronize the cached data to the disk, the request is handled by a kernel thread, whose workload will not be accounted to the container. According to the experiments[2] in the Docker container environment, this attack can reduce the performance of other containers by 85% to 95%.

2) Service processes. There are many service processes in the Linux operating system to provide system-level services, such as security audits, system logging, etc [2]. When a specific event is triggered, the corresponding service process will handle it. These processes do not belong to the container cgroups, so the resources consumed by them will not be charged to the containers.

For instance, to generate workloads into service processes from containers, attackers can leverage the journald service, which collects system logs. When some events (e.g., switching users, adding users/groups, exceptions, etc.) are triggered in containers, these events are logged into system logs by the journald service, consuming the host resources. Since the journald service process does not belong to the container's control groups (cgroups), the resources consumed by its logging activities are not counted towards the container's resource usage. This approach indirectly strain the host, potentially leading to decreased performance or disruptions on the host.

3) Interrupt context. The resources consumed in the interrupt context will not be accounted to any processes. When an interrupt is raised, the software interrupt handlers will be scheduled to handle the softirq. If the workload is too heavy, it will be offloaded to a kernel thread, which will also not be accounted to the original cgroups. Thus, if a container can trigger lots of softirqs, it can break the cgroup limitation. Even though there are many kinds of softirqs, containers are only easy to trigger the network and block softirqs by sending packages and writing files. The resources of handling these softirqs are not accounted to the containers.

B. Observation

These escape attacks exploit the underlying mechanisms of the Linux kernel (e.g., kernel threads, service processes, and the interrupt context), which are difficult to redesign and implement to solve the problem. In addition, most of the interfaces and underlying Linux mechanisms that attacks rely on are necessary for generic scenarios and are difficult to defend against by blocking several specific interfaces[2]. Furthermore, it is challenging to detect attack behaviors due to the similarity between normal behaviors and attack behaviors. It is normal for a container to use attack-related mechanisms (e.g., kernel threads, etc.). For instance, a normal network-intensive process also needs to send network messages frequently, so it is not easy to determine if it is an attack process. In addition, the attack behaviors may hide in the normal behaviors. For example, attackers may perform attack behaviors while providing normal services. It is difficult to determine a fixed behavior frequency or interval threshold for identifying attacks.

With the development of deep learning, it is possible to identify anomaly behaviors or attacks from container be-

haviors (e.g., system calls). However, the current learning-based anomaly detection approaches do not perform well in dynamic container behavior analysis [7]. That is because these approaches[10, 11, 12, 13, 14, 15] mainly focus on the behavior sequence and frequency, which are not enough for container scenarios, especially to detect the cgroups escape attacks. Firstly, the same operations with different targets and arguments will cause different effects. For instance, writing to system logs or files will introduce different workloads to the system services, and the data size will also affect the introduced workload. So, only analyzing the operation sequences cannot identify the attacks. Second, attack behavior sequences might include a lot of normal behaviors because a wide variety of unknown programs are running in the cloud environment, each with diverse and unpredictable behavior patterns. This makes it difficult to set a fixed behavioral frequency or interval threshold to identify attacks.

Based on our observation, a program execution consists of a series of behavior sets, and each behavior set has a behavior intention. Behavior intention features are able to describe the overall semantics of a series of associated behaviors, including behavior goals, operating target, behavior parameters, duration, etc. By correlating behaviors that have the same intention, it is possible to better distinguish attack behaviors from other behaviors in a sequence of behaviors. Since behavior intention features can comprehensively reflect the overall intention and semantics of local behaviors, behavior context analysis based on behavior intention features can well learn the characteristics of local and overall behaviors so as to analyze their security. Therefore, we aim to propose a cgroups escape detection approach for containers based on the modeling and analysis of behavior intention features.

III. SYSTEM OVERVIEW AND DATA PREPARATION

A. System Overview

To detect the cgroups escape attacks, we propose EsCapturer, a novel container behavior analysis approach based on behavior-intention learning, whose architecture is shown in Figure 1. Firstly, the attack-related container behaviors are captured by the behavior interceptor. The monitored events are selected based on the analysis of container cgroups escape attacks so that the interceptor can obtain all of the attack-related behaviors with low overhead. The interceptor runs in the host kernel, so it is transparent to the monitored containers. Next, the behavior logs are correlated by the correlation module, which leverages a density-based method to correlate possible related events and further identifies behavior patterns to correlate large-scale logs. The correlated behaviors are used to train the deep learning network for anomaly detection based on behavior-intention learning.

Overall, EsCapturer has three parts, including: the container behavior monitor (Section III-B), the behavior correlation (Section III-C) and the deep-learning-based attack detection model (Section IV).

B. Monitoring Container Behaviors

To detect the cgroups escape attacks, EsCapturer needs to capture all of the attack-related container behaviors. As

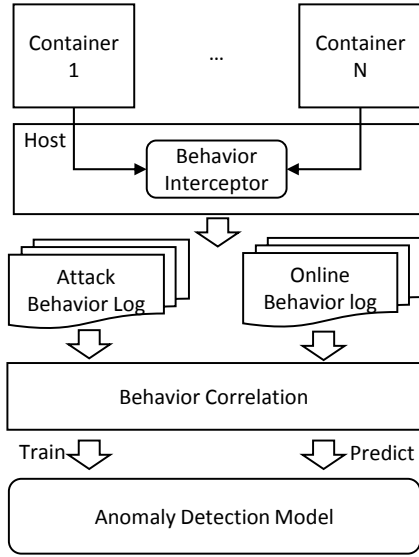


Fig. 1. The system architecture of EsCapturer.

described in Section II, the core idea of the attacks is to inject workloads into other system components by exploiting some underlying mechanisms. Therefore, we need to analyze these mechanisms and the corresponding attack vectors to find out the attack-related container events to be monitored, which are as follows.

1) Kernel upcall APIs. To exploit the kernel upcall mechanism, attackers need to trigger kernel threads and then let the threads upcall other user-space processes. Containers can trigger exceptions by dividing zero repeatedly, and the exception handler will use the kernel upcall API to inject workloads into user-space processes. To capture all of this kind of behaviors, the usermode helper APIs (e.g., `call_usermodehelper_setup`, `call_usermodehelper_exec`, `call_usermodehelper_exec_async` and `call_usermodehelper_exec_work`) of the Linux kernel should be monitored.

2) Kernel thread APIs. The workloads of kernel threads are not accounted to the container cgroups, so attackers can leverage the related kernel threads to consume the host resources. So, the kernel thread APIs (e.g., `queue_work_on`, `__queue_work` and `queue_delayed_work_on`) are monitored by EsCapturer.

3) Critical syscalls. Attackers can inject workloads into some system service processes (e.g., `journald`) by triggering some specific events (e.g., switching users, adding users/groups) in containers. These events will send messages to the system log service. To capture them, the network-related syscalls (e.g., `sendto`, `sendmsg`, etc.) are monitored. Furthermore, containers can leverage file-related and network-related syscalls to trigger the softirqs, which can inject workloads into the interrupt context. So, the file-related syscalls (e.g., `read`, `write`, etc.) are monitored.

Overall, all of the monitored events are kernel-underlying API entries related to the vulnerable mechanisms, so they are able to capture the corresponding container behaviors. The mapping between the attack behaviors and the corresponding

TABLE I
KERNEL EVENTS MONITORED BY ESCAPTURER.

Attack Methods	Mech.	Monitored Events
Triggering exceptions	KU	Kernel upcall APIs
Write-back syscalls	KT	Kernel threads APIs
Writing to terminals	SP	Kernel threads APIs
Triggering critical events	SP	Critical syscalls
File/network operations	IC	Critical syscalls

EU=Kernel Upcalls; KT=Kernel Threads; SP=Service Processes; IC=Interrupt Context.

monitored events is shown in Table I.

To monitor these kernel events, some critical kernel functions should be monitored by EsCapturer. Therefore, a kernel module is developed in the host kernel, which hooks the corresponding kernel functions and logs the events. Only the processes or threads belonging to the target containers are monitored. For every event, the timestamp, parameters, and thread information are recorded. Since the behavior interceptor runs in the kernel, it is transparent to the containers.

C. Behavior Correlation

To analyze behavior intentions, it is necessary to correlate related behaviors that perform the same behavior intention. Typically, related behaviors have the same operating targets. Based on this observation, related behaviors can be identified. However, there are two challenges in correlating behaviors. Firstly, only analyzing the parameters cannot find the behavior relationships. For instance, two syscalls using the same file descriptor may not be related because the file descriptor may be released and recreated between the two syscalls. However, if it is not released, these two syscalls are related. In addition, some syscalls may have the same parameter value, but the parameters may have different meanings. For instance, the parameter of a kill syscall may be the same as that of an open syscall, but they are not related. So, how to correlate behaviors systematically is necessary.

To address these challenges, a novel automated two-stage correlation approach is proposed, whose workflow is shown in Figure 2. Since the thread information is recorded in the logs, the behavior logs are first organized by threads. So, the context switches will not affect the analysis. In general, there are two stages of behavior correlation, including the parameter-based correlation and the density-based correlation. The parameter-based correlation aims to find out all of the possible related events, and the density-based correlation further identifies high-density behaviors. Finally, the correlation rules are generated to correlate behaviors in large-scale behavior logs.

1) Parameter-based correlation. EsCapturer first identifies all of the events with the same parameters, based on the observation that the syscalls operating on the same target usually have the same parameter (i.e., file descriptor). Since the file descriptors are usually the first parameters of syscalls, only the first parameters are analyzed. Next, the corresponding initialization syscalls (e.g., `open`, `bind`, etc.) are identified. Through our observation, the return values of initialization syscalls are usually the file descriptors, which are used by

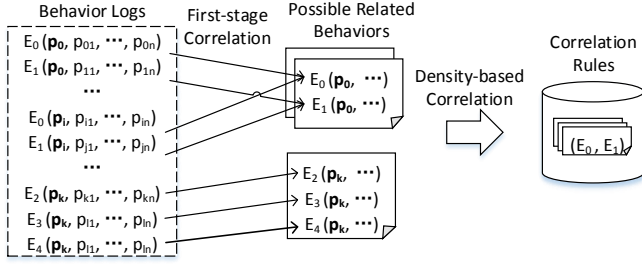


Fig. 2. The workflow of behavior correlation.

TABLE II
CO-OCCURRENCE MATRIX OF BEHAVIOR SETS.

	S_1	...	S_j	...	S_m
I_1	$P(1, 1)$		$P(1, j)$		$P(1, m)$
...					
I_i	$P(i, 1)$		$P(i, j) = \frac{f(I_i, S_j)}{f(I_i)}$		$P(i, m)$
...					
I_n	$P(n, 1)$		$P(n, j)$		$P(n, m)$

the related syscalls. So, the system call that is before the correlated behaviors is considered as the corresponding initialization syscall, and the return value is used as the first parameter of the behaviors. The initialization syscalls split the correlated behaviors into behavior sets. After that, the correlated behaviors are used to generate the corresponding automata. For instance, the set “open, read, read, close” will be processed into the automata “open, read, close”. These behavior sets are possible related syscalls, but the parameter-based correlation cannot identify the irrelevant syscalls with the same parameter value.

2) Density-based correlation. The density-based correlation can further identify strongly related behaviors. Based on the observation that real related behaviors usually appear together but irrelevant behaviors may not always appear together and the orders are not fixed, EsCapturer can find out behavior sets with a high co-occurrence rate. To that end, EsCapturer counts the number of every unique behavior set and removes the low-density behavior sets. We build a co-occurrence matrix as shown in Table II, where each element $P(i, j)$ means the proportion of behavior set S_j in all behavior sets containing the initialization behavior I_i . Specifically, let $f(I_i, S_j)$ be the number of S_j that contains the initialization behavior I_i . If S_j does not include I_i , $f(I_i, S_j)$ will be zero. Otherwise, it will be the number of occurrences of S_j . And, $f(I_i)$ represents the number of all behaviors sets that contains the initialization behavior I_i . Therefore the $f(I_i, S_j)$ describes the frequency of S_j in I_i . To identify the high-density behavior sets, we check if the $P(i, j)$ is greater than a threshold τ (say $\tau = 0.1$). If so, the behaviors are considered to be related.

3) Large-scale behavior correlation. Based on the high-density behavior sets, the corresponding automata are generated, which are used for large-scale behavior log processing. When a behavior log fits the start state of an automata, EsCapturer can extract all the related logs that fit the automata. After that, the extracted related behaviors can be analyzed by the deep-learning model.

IV. DEEP-LEARNING-BASED ANOMALY DETECTION

EsCapturer utilizes a multi-modal learning approach, employing both the sequential and graph structure features of behavior intention from logs to detect cgroups escape attacks. The framework of our approach, as shown in Figure 3, consists of four main steps: graph feature modeling, sequential feature modeling, heterogeneous feature fusion, and anomaly detection.

EsCapturer initiates by extracting multiple events with the same behavior intention from the logs. Each of the multiple events can be constructed into a behavior intention graph. These graphs are then encoded into vectors via graph embedding techniques to represent the dependency relationships between multiple events. Concurrently, we focus on learning the sequential features of the multiple events, which provide a semantic representation of the behavior intention. In addition to utilizing graph embedding and sequence embedding for learning behavior intentions, we also introduce two Transformer encoders, one for learning the contextual features of graph embedding and the other for learning the contextual features of sequence embedding, to further extract the correlations between behavior intentions. The core of our framework also involves an attention-based fusion module to merge these two features, achieving a comprehensive global representation. This global representation is then utilized by a Deep Single-Class Support Vector Data Description (DSVDD) model to establish a decision boundary for the detection of container resource escape attacks, enhancing the system’s capability to identify attack patterns effectively.

A. Sequential Feature Modeling

For a set of log messages $L = \{l_1, l_2, \dots, l_n\}$, they are first grouped based on similar behavior intentions, forming a collection of log groups $G = \{G_1, G_2, \dots, G_m\}$.

In EsCapturer, the representation of behavior intentions is derived by extracting the sequential feature of events within each log group. The frequency and the type of events, especially in the context of container escape attacks, are critical indicators of malicious behaviors. To achieve this, EsCapturer calculates and normalizes the frequencies of log event types, with $c(l_j)$ denoting the normalized frequency for event type j . Additionally, EsCapturer employs Word2Vec for generating vectors for each event type and its parameters, with $d(l_j)$ representing the vector of event type j . EsCapturer concatenate $c(l_j)$ and $d(l_j)$ to form a comprehensive representation of event:

$$x_j = [c(l_j); d(l_j)] \quad (1)$$

Next, the composite representations of all events within each log group G_i are input as a sequence into the Transformer encoder network to extract the semantic information of the events within the behavior intention. For the sequence of composite event representations $X_i = \{x_{i1}, x_{i2}, \dots\}$ in the log group G_i , after being processed by the Transformer encoder, we obtain the transformed hidden state vectors h_{si} :

$$h_{si} = \phi_{\text{transformer}}(X_i) \quad (2)$$

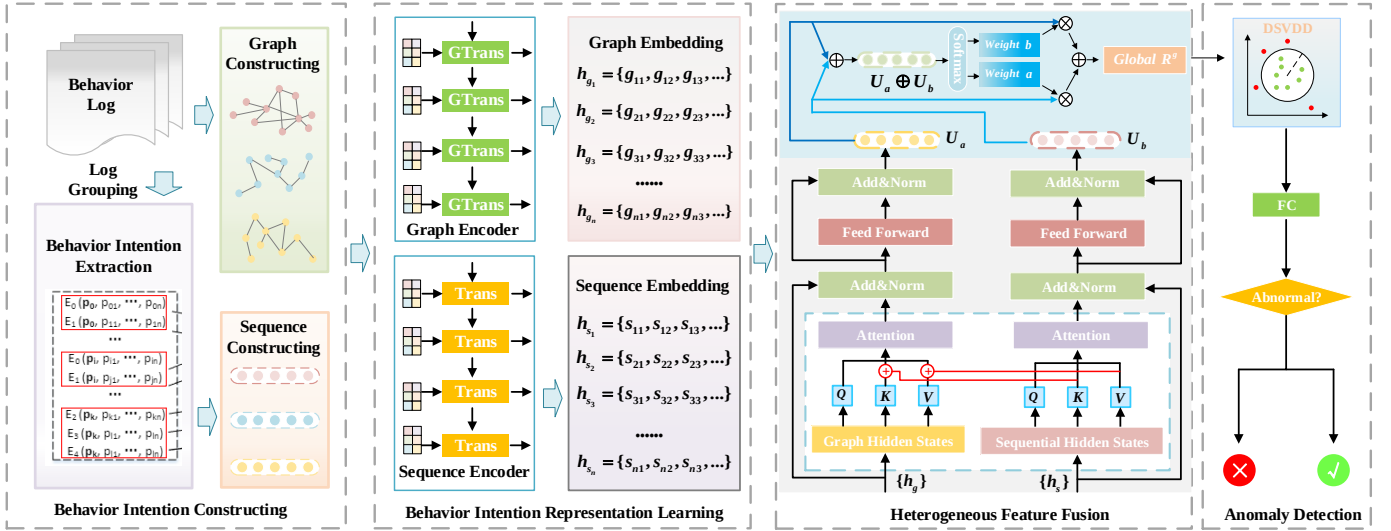


Fig. 3. The behavior sequences obtained from tracking are processed through Graph Encoders and Sequence Encoders to generate Graph Embedding and Sequence Embedding. These embeddings are then fused using a cross-attention mechanism to integrate features to perform detection.

Thus, the sequential embeddings of all behavior intentions in log can be represented as $h_s = \{h_{s1}, h_{s2}, \dots\}$. This method leverages the Transformer's ability to handle complex dependencies within sequences, thus enabling the extraction of rich, contextual semantic information from a series of event embeddings.

B. Graph Feather Modeling

1) Behavior Intention Graph Construction. To enhance the detection of Cgroups escape attacks, we first construct behavior intention graphs for each of the multiple events with the same behavior intention, where both nodes and edges are represented as vector features. This focuses on capture dependencies within the events.

For each log group $G_i \in G$, we construct a graph $\mathcal{G}_i = (\mathcal{V}_i, A_i, X_{v_i}, F_{w_i})$. Where, $\mathcal{V}_i$ is the set of nodes representing the events in G_i , A_i is the adjacency matrix of edges representing the interactions between these events, $X_{v_i} \in \mathbb{R}^{|\mathcal{V}_i| \times d}$ is the matrix of event embeddings, and $F_{w_i} \in \mathbb{R}^{|A_i| \times p}$ is the matrix of edge feature vectors, where d and p are the dimensions of the event embeddings and edge feature vectors, respectively.

In each graph, each node represents an event from the log group and is characterized by an embedding vector using the Word2Vec algorithm.

Edges in $\mathcal{G}_i$ represent the interactions characters between events within the group G_i . Instead of using simple weights (on the occurrence frequency of consecutive events in the sequence), each edge $a \in A_i$ is associated with a vector $e^{j,k}$ that encapsulates more nuanced information about the relationship between nodes v_j and v_k . It includes various features, including the frequency of occurrence, cumulative time duration between events, and the sequential order of these events.

Through this construction, each event graph $\mathcal{G}_i$ becomes a rich representation of the dependency relationships between events within a specific behavior intention. The vector-based approach for both nodes and edges allows for a more detailed

and informative representation, enabling more effective identification of anomalous behaviors that could signify container escape attacks.

2) Behavior Intention Graph Representation Learning. We have made customized improvements to the graph transformer to achieve representation learning for behavior intention graphs.

The Graph Transformer Layer. The D -layer graph transformer will aggregate information from the event graph to learn the target node representation. Each graph transformer layer contains three trainable matrices: $W^Q, W^K, W^V \in \mathbb{R}^{d_h \times d_h}$, and queries Q , keys K , and values V are generated by multiplying the hidden layer input $H^{(l-1)}$ correspondingly with W^Q, W^K and W^V :

$$\{Q, K, V\} = H^{(l-1)} \otimes \{W_Q^{(l)}, W_K^{(l)}, W_V^{(l)}\}. \quad (3)$$

Then the l -th G-Transformer layer can be denoted as:

$$H^{(l)} = \text{G-Transformer} \left(H^{(l-1)} \right) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_h}} \right) V \quad (4)$$

The layer update equations for a layer ℓ is:

$$\hat{h}_i^{\ell+1} = O_h^\ell \parallel \left(\sum_{k=1}^M w_{ij}^{k,\ell} V^{k,\ell} h_j^\ell \right), \hat{h}_i^0 = x_i \quad (5)$$

$$\hat{e}_{ij}^{(\ell+1)} = O_e^{(\ell)} \parallel (\hat{w}_{ij}^{k,\ell}) \quad (6)$$

$$w_{ij}^{k,\ell} = \text{softmax}_j(\hat{w}_{ij}^{k,\ell}) \quad (7)$$

$$\hat{w}_{ij}^{k,\ell} = \frac{Q^{k,\ell} \hat{h}_i^\ell \cdot K^{k,\ell} h_j^\ell}{\sqrt{d_k}} \cdot E^{k,\ell} e_{ij}^{\ell} \quad (8)$$

The matrices $Q^{k,\ell}, K^{k,\ell}, V^{k,\ell}, E^{k,\ell} \in \mathbb{R}^{d_k \times d}$, $O_h^\ell, O_e^\ell \in \mathbb{R}^{d \times d}$. Here, $k = 1$ to M indicates the count of attention heads, with $\parallel$ representing concatenation. The outputs $\hat{h}_i^{\ell+1}$

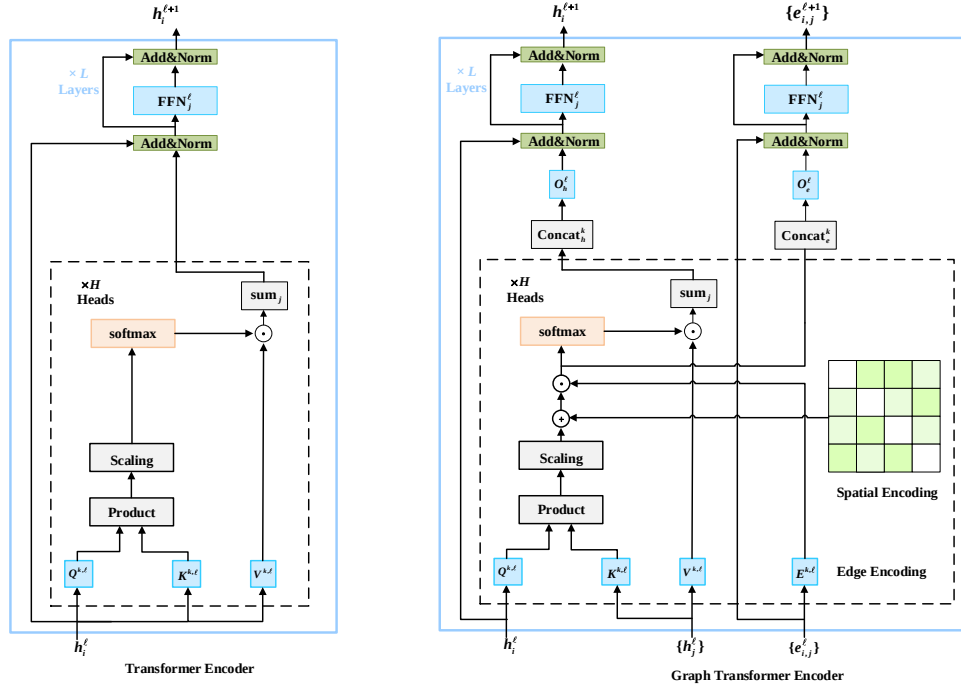


Fig. 4. Graph Transformer Layer with edge features.

and $\hat{e}_{ij}^{\ell+1}$ proceed through different Feed Forward Networks, each framed by preceding residual connections and subsequent normalization layers.

Spatial Encoding. Unlike sequential data that can be encoded using positional embeddings, graph structures present a unique challenge due to their non-sequential nature. Graphs exist in multi-dimensional spatial spaces with nodes interconnected by edges, necessitating a different approach for encoding. To address this, we also employ a spatial encoding technique specifically designed to embed and capture the structural information of graphs effectively.

Specifically, for a given graph $\mathcal{G}$, we define a function $\phi(v_i, v_j) : V \times V \rightarrow \mathbb{R}$ that quantifies the spatial relationship between vertices v_i and v_j in $\mathcal{G}$. This function, ϕ , is determined by the connectivity of the nodes and is assigned the shortest path distance (SPD) between v_i and v_j when the nodes are connected. Otherwise, $\phi(v_i, v_j)$ is set to a predefined special value, notably -1. Each potential output of ϕ is associated with a learnable scalar, acting as a bias in the self-attention mechanism. Let P_{ij} represent the element at position (i, j) in the Query-Key product matrix, we have:

$$P_{ij} = \frac{(h_i W^Q)(h_j W^K)^T}{\sqrt{d}} + b_{\phi(v_i, v_j)} \quad (9)$$

where $b_{\phi(v_i, v_j)}$ is a learnable scalar indexed by $\phi(v_i, v_j)$, and shared across all layers.

Thus, we can add spatial encoding to Graph Transformer and Equation 8 can be rewritten as:

$$\hat{w}_{ij}^{k, \ell} = \left(\frac{Q^{k, \ell} \hat{h}_i^{\ell} \cdot K^{k, \ell} h_j^{\ell}}{\sqrt{d_k}} + b_{\phi(v_i, v_j)} \right) \cdot E^{k, \ell} e_{ij, \ell}^{j, \ell} \quad (10)$$

Graph Embedding. At the final layer D , we obtain the embedding $\hat{h}_i^D$ for each node i . To aggregate this node-level information into a comprehensive representation of the graph $\mathcal{G}$, all node embeddings $\hat{h}_i^D$ are aggregated to generate a holistic representation h_g of the graph. Mathematically, this process is described as follows:

$$h_g = \frac{1}{Z} \sum_{i=1}^N \sum_{j \in \mathcal{N}_i} \hat{w}_{ij} \cdot \hat{h}_j^D \quad (11)$$

Here, $\hat{w}_{ij}$ represents the node-to-node weights optimized through the self-attention mechanism, indicating the strength of interaction between nodes i and j ; N is the total number of nodes in the graph; $\mathcal{N}_i$ denotes the set of neighboring nodes of node i ; Z is a normalization constant ensuring the stability of the graph embedding, defined as:

$$Z = \sum_{i=1}^N \sum_{j \in \mathcal{N}_i} \hat{w}_{ij} \quad (12)$$

Thus, the graph structure embeddings of all behavior intentions in log can be represented as $h_g = \{h_{g1}, h_{g2}, \dots\}$.

C. Heterogeneous Feature Fusion

We then utilize two Transformer Encoder networks to process and fuse the graph-structured embeddings h_g and sequential feature embeddings h_s of each behavior intention, as shown in Fig.3. The aim is to capture the semantic information between multiple behavior intentions and to effectively combine heterogeneous information for a holistic representation.

The standard attention mechanism in the Transformer Encoder is used:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (13)$$

where Q , K , and V represent the query, key, and value matrices, respectively, and d_k is the scaling factor. For sequential features, this mechanism is specifically represented as:

$$\text{head}M_t = \text{Attn}(h_s W_s^q, h_s W_s^k, h_s W_s^v), \quad (14)$$

where h_s represents the sequential feature representation, and W_s^q , W_s^k , W_s^v are the query, key, and value weight matrices, respectively.

In the Transformer Encoder for the graph-structured features, the attention mechanism is uniquely adapted to integrate the key (K) and value (V) from the sequential feature representations:

$$\text{head}M_v = \text{Attn}(h_g W_g^q, [h_g W_g^k, h_s W_s^k], [h_g W_g^v, h_s W_s^v]) \quad (15)$$

where h_g denotes the graph-structured feature representation. In this configuration, the key and value components from the serialized feature's Transformer Encoder ($h_s W_s^k$ and $h_s W_s^v$) are concatenated with those of the graph-structured features. This design allows the graph-structured feature's Transformer to access and leverage the contextual information embedded in the serialized features, enriching its understanding and analysis of the graph-structured data.

The outputs U_a and U_b from these two transformer encoder networks, are subsequently merged, creating a unified and enriched representation. The fusion process is described as follows:

First, we fuse results from multiple branches via an element-wise summation:

$$U_c = \text{concat}(U_a, U_b) \quad (16)$$

For the features U_a and U_b of length L , we embed global information by applying global average pooling:

$$p = \frac{1}{L} \sum_{i=1}^L U_{c_i} \quad (17)$$

A compact feature $z \in \mathbb{R}^{d \times 1}$ is then created to guide the precise and adaptive selections, using a fully connected (fc) layer:

$$z = F_{\text{fc}}(p) \quad (18)$$

Next, a soft attention across channels is applied, guided by the compact feature descriptor z :

$$a_i = \frac{e^{A_i z}}{e^{A_i z} + e^{B_i z}}; \quad b_i = \frac{e^{B_i z}}{e^{A_i z} + e^{B_i z}}; \quad (19)$$

where $A, B \in \mathbb{R}^{L \times d}$ and a, b are the soft attention vectors for U_a and U_b , respectively. The final feature map V is obtained through attention weights:

$$U_{d_i} = \text{concat}(a_i \cdot U_{a_i}, b_i \cdot U_{b_i}); \quad a_i + b_i = 1 \quad (20)$$

where $U_d = [U_{d_1}, U_{d_2}, \dots, U_{d_L}]$, $U_d \in \mathbb{R}^{L \times D}$.

The final global representations can be defined as:

$$h_d = \frac{1}{L} \sum_{i=1}^L U_{d_i} \quad (21)$$

D. Anomaly Detection

Deep Support Vector Data Description (DSVDD) is utilized to detect anomalies by evaluating the feature representations of container behavior. DSVDD is trained on anomalous data to learn distinct representations of malicious behaviors, enabling the model to effectively identify and differentiate anomalies in new data based on their deviation from the learned norm. The One-Class Deep SVDD objective is formally defined as:

$$\min_{\mathcal{W}} \frac{1}{n} \sum_{i=1}^n \|\phi(x_i; \mathcal{W}) - c\|^2 + \frac{\lambda}{2} \sum_{\ell=1}^L \|W^\ell\|_F^2 \quad (22)$$

where $h_d = \phi(x; \mathcal{W}) \in \mathcal{F}$ is the feature representation of x given by EsCapturer ϕ with parameters $W \in \mathcal{W}$; c is the center of the hypersphere in output space $\mathcal{F}$. The last term $\frac{\lambda}{2} \sum_{\ell=1}^L \|W^\ell\|_F^2$ is a weight decay regularizer on the network parameters $\mathcal{W}$ with hyperparameter $\lambda > 0$, where $\|\cdot\|_F$ denotes the Frobenius norm.

After training the model on abnormal behavior data, the anomaly score for a new test data point U_m is computed as:

$$\text{score}(U_m) = 1/\|\phi(U_m; \mathcal{W}) - c\|^2. \quad (23)$$

This formulation enables the identification of anomalous behaviors by evaluating the proximity of a new data point's transformed feature representation to the center of the hypersphere in the learned feature space. The points closer to the center are considered more anomalous.

V. IMPLEMENTATION

A. Monitoring Kernel Events

EsCapturer is implemented based on Sysdig[16], which is a syscall capture tool for Linux kernels. We leverage Sysdig to intercept 23 types of system call events in order to monitor related kernel events comprehensively with low monitoring overhead. Sysdig provides a pre-built kernel module that captures these events efficiently, eliminating the need for custom kernel module development. As described in Section III-B, we also need to intercept kernel functions to capture the kernel upcall APIs (e.g., `call_usermodehelper`). EsCapturer captures seven types of kernel functions with the tool `trace-cmd`[17], which is the front-end application of `ftrace`[18] built in Linux kernels above 2.6.27. Sysdig and `ftrace` together provide a comprehensive solution for capturing both system calls and kernel functions efficiently. Besides, the logs recorded by Sysdig and `trace-cmd` are merged according to the timestamp.

B. Container Process Identification

The Linux namespaces technology is used to isolate different containers on a host. The processes of a container share the same namespaces, so the namespace fields of process structures (i.e., `task_struct`) can be used to identify container processes. Based on this observation, EsCapturer first identifies the PID namespace of the target container to protect it, then intercepts the events of processes or threads with the same namespace. Every event log contains thread information, so there is no need to consider the problem of multi-threading in

TABLE III
DOCKER IMAGES WITH THE CORRESPONDING BENCHMARK TOOLS.

Docker images	Benchmark tools
MongoDB	mongo-perf
PostgreSQL	pgbench
Redis	redis-benchmark
Memcached	mcperf
Nginx	httperf

the process. Based on the PID namespace, logs that belong to the same container can be correlated and analyzed together. By monitoring logs online, EsCapturer can detect attacks in a timely manner before they cause harm.

VI. EVALUATION

We evaluate EsCapturer by answering the following research questions (RQs):

- RQ1: How effective is EsCapturer in anomaly detection of container behaviors?
- RQ2: What is the contribution of each design of EsCapturer?
- RQ3: What is the impact of the monitoring overhead introduced by EsCapturer?

A. Datasets

Normal Behaviors. In order to evaluate the impact of cgroups escape attacks and generate a dataset of normal behavior logs, we select the current mainstream Docker images on Docker Hub and leverage the corresponding available benchmark tools to generate normal execution workloads. For example, for MongoDB, we use mongo-perf[2] and apply all the test cases, such as simple_GEO and simple_query. The other Docker images and the corresponding test tools are shown in Table III. By using these tools to generate the container workloads, the corresponding behavior logs can be captured by the behavior interceptor.

Attack Behaviors. The attack behaviors are performed in a real Docker environment following the attack vectors proposed by [2], which exploit container resource isolation through several mechanisms.

- 1) Kernel upcalls. This mechanism can be exploited by triggering exceptions, such as faults and traps, etc.
- 2) Kernel threads. We perform the attack by introducing workloads to kernel threads, such as I/O requests and data synchronization, etc.
- 3) Service processes. The workloads are introduced by triggering log-recording events, such as generating logs, etc.
- 4) Container engine. The additional workloads can be introduced by outputting data to the terminal of the container.
- 5) Softirq handling. We perform I/O requests to increase the overhead caused by the soft interrupt context.

These attacks can consume resources on the host, thus affecting the execution of other containers or even enabling DoS attacks. Some of these system services are not commonly used by container instances, such as kernel upcalls, but most of them, such as kernel threads, container engines, are commonly

TABLE IV
ATTACK INTERVAL WHEN EFFECT EXCEEDS 10%.

Attack Mechanisms	Maximal Interval
Service processes	0.234s
Kernel upcalls	0.09s
Softirq handling/Container engine	0.005s
Kernel threads	0.001s

applied by container instances. In order to make the behavior data closer to the real scenario, we perform the attack behaviors during normal behaviors. To verify the attack effect, we test the attack effect in our local Docker environment. To that end, we first use the cgroups to limit the available CPU resources of two containers to 50%. In normal cases, each container can consume at most 50% of the CPU resource, and they cannot affect each other. We perform the cgroups escape attacks in one container and constantly change the proportion of attack behaviors (i.e., attack frequency). During this process, we measure the impact on the performance of another container. Although the effectiveness of the attack decreases as the time interval between attacks becomes longer, the attack container is still able to influence the execution of other containers. At the attack intervals shown in Table IV, the attack effect can still be greater than 10%. If multiple attack methods are used or multiple containers are attacked at the same time, the attack effect will be more obvious.

In our experiment, the attack frequency is selected randomly higher than the values shown in Table IV for each attack. We leverage the behavior interceptor to record the behavior logs during the attacks. Finally, we collected 2,252,225 benign log records and 615,227 attack log records containing five types of attacks¹.

B. Baselines and Metrics

We compare our approach with representative baselines, including well-established methods introduced in [7] and several recent state-of-the-art techniques, such as:

1) Liao et al. [10]. This approach utilizes a weighted Relaxed SVM. It extracts dynamic features from system call sequences using TF-IDF and suffix trees, and applies a cost-sensitive classifier to address class imbalance.

2) Subba et al. [11]. In this method, system call sequences are converted into TF-IDF weighted n-gram vectors, reduced via truncated SVD, and classified using a decision tree to detect anomalies.

3) Shamim et al. [12]. A centroid-based vector-space model is employed, where system call sequences are represented as TF-IDF vectors and anomaly detection is performed by computing the distance from a normal centroid.

4) Guan et al. [13]. This method adopts a hybrid architecture of LSTM and Transformer. Count vectors derived from system call sequences are input to the model, where LSTM captures temporal dependencies and Transformer handles global context.

¹We will release the dataset and source code after the acceptance.

5) **Abderrahmane et al. [14]**. CNN is leveraged in this work to analyze system call dependencies represented as matrices, enabling effective Android malware detection through spatial pattern recognition.

6) **Joomye et al. [15]**. The authors propose a Temporal Convolutional Network (TCN)-based model that processes system calls, binder calls, and composite behaviours separately, and integrates them for dynamic Android malware detection.

We use Precision, Recall and F1-Score to evaluate the performance of EsCapturer:

- Precision: the percentage of logs that are correctly identified as anomalies over all of the logs that are identified as anomalies:

$$Precision = \frac{TP}{TP + FP}$$

- Recall: the percentage of logs that are correctly identified as anomalies over all of the abnormal log sequences:

$$Recall = \frac{TP}{TP + FN}$$

- F1-Score: the harmonic mean of Precision and Recall:

$$F1 - Score = \frac{2 * Precision * Recall}{Precision + Recall}$$

TP is the number of abnormal logs correctly detected by the model. FP is the number of normal logs incorrectly identified as anomalies by the model. FN is the number of abnormal logs not detected by the model.

C. Performance of Anomaly Detection (RQ1)

We used 75% of the data sets for training, and 25% of the data sets are used for testing. Table V presents a comparative analysis of the overall performance of various methods. Our approach, EsCapturer, outperforms all baseline methods, achieving the best results for each type of attack. In terms of F1 scores, EsCapturer reaches an average of 98.56% in five attacks, surpassing other methods by 6.2% to 14.2%. The high recall and precision scores of EsCapturer indicate its proficiency in detecting anomalies with minimal false alerts. Thus, EsCapturer is highly effective in detecting Cgroups attacks.

The success of EsCapturer can be attributed to three main factors: Firstly, EsCapturer learns behavior intent features from logs. These features comprehensively reflect the overall intent and semantics of local behaviors, enabling effective analysis of malicious behaviors of cgroups attacks. In contrast, other methods merely transform logs into sequences of events, facing challenges in extracting meaningful behavior semantics, especially in the presence of hundreds of events. Secondly, the superiority of EsCapturer over other methods is also due to its effective utilization of two types of feature representations of behavior intentions: serialized features and graph structural features. This enables EsCapturer to capture both the semantic information of behavior intents and the dependencies among events within an intention. Additionally, EsCapturer employs a meticulously designed fusion approach to integrate these two types of features deeply.

Overall, EsCapturer effectively detects system anomalies across all datasets. Compared with all baselines involved in each evaluation metric, it significantly enhances effectiveness.

D. Ablation Experiments (RQ2)

We conduct an extensive ablation study on EsCapturer. We investigate the contribution of the introduction of behavior-intention learning, which includes Event Graph Feather Modeling, Sequential Feature Modeling, and Heterogeneous Feature Fusion. Through these ablation experiments, we aim to dissect the individual contributions of various components in EsCapturer.

Removal of Sequential Feature of Behavior Intention (ro-S): In this ablation, we remove the sequential feature representation of behavior intention from EsCapturer. This feature typically encapsulates the sequential aspect of behavior, presenting semantic information of behavior intention. By excluding it, we aim to assess how the absence of sequential data representation affects detection efficacy.

Removal of Graph Structure Feature of Behavior Intention (ro-G): Another crucial aspect of our ablation study involves the removal of the graph structure feature of behavior intention. This feature represents the complex inter-relationships and dependencies among various events within a behavior intention. By conducting an analysis without this feature, we aim to evaluate the significance of the relational information provided by the graph structure.

Removal of Behavior Intention (ro-I): In this ablation, we directly exclude the consideration of behavior intention from EsCapturer. Specifically, we focus solely on extracting sequential features related to the timing of behaviors without incorporating any semantic information about behavior intention. This experiment aims to evaluate the impact of disregarding behavior intention on detection efficacy.

The results are shown in Table VI, which are discussed as follows:

(1) Removal of Sequential Feature of Behavior Intention: The ablation study demonstrated a substantial impact on EsCapturer's performance due to the removal of the serialized feature. Specifically, The F1-score has decreased by 1.4% on average in five types of attacks, highlighting the critical impact of sequential features on the EsCapturer. The absence of this sequential representation led to a reduction in the model's ability to detect complex behavior patterns.

(2) Removal of Graph Structure Feature of Behavior Intention: Omitting the graph-structured feature resulted in a 0.8% decrease in the anomaly detection rate. This reduction points to the crucial role that relational information plays in the comprehensive analysis of behavior intentions.

(3) Removal of Behavior Intention: The ablation study demonstrated a substantial impact on EsCapturer's performance due to the removal of behavior intention features. The absence of this information led to a reduction in the model's ability to detect complex behavior patterns, resulting in an average decrease of 2.18% in the F1-score across all evaluated attack types. This implies that by incorporating behavior intention features, EsCapturer can effectively identify

TABLE V
THE PERFORMANCE OF VARIOUS METHODS IN DETECTING DIFFERENT ATTACKS

Method	Kernel Upcalls			Kernel Threads			System Services			Container Engine			Softirq Handling		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Liao [10]	0.982	0.773	0.865	0.881	0.845	0.863	0.987	0.878	0.930	0.989	0.777	0.870	0.865	0.991	0.924
Subba [11]	0.972	0.792	0.872	0.883	0.864	0.874	0.956	0.888	0.921	0.975	0.823	0.892	0.885	0.963	0.922
Shamim [12]	0.956	0.749	0.840	0.898	0.824	0.860	0.920	0.870	0.894	0.993	0.781	0.874	0.983	0.885	0.932
Guan [13]	0.968	0.790	0.870	0.965	0.817	0.885	0.997	0.870	0.929	0.984	0.828	0.900	0.867	0.988	0.923
Abderrahmane [14]	0.948	0.824	0.881	0.932	0.828	0.877	0.990	0.880	0.932	0.973	0.844	0.904	0.864	0.991	0.923
Joomye [15]	0.964	0.801	0.875	0.896	0.862	0.879	0.988	0.879	0.930	0.972	0.845	0.904	0.874	0.976	0.922
EsCapturer	0.976	0.987	0.982	0.972	0.990	0.981	0.998	0.998	0.998	0.974	0.992	0.983	0.980	0.988	0.984

TABLE VI
ABLATION RESULTS ON DETECTION OF FIVE TYPES OF ESCAPE ATTACKS

Attack type	Metric	ro-S	ro-G	ro-I
Kernel Upcalls	Precision	0.964	0.970	0.930
	Recall	0.982	0.985	0.987
	F1-Score	0.973	0.977	0.958
Kernel Threads	Precision	0.946	0.961	0.946
	Recall	0.984	0.987	0.972
	F1-Score	0.965	0.974	0.959
System Services	Precision	0.989	0.995	0.976
	Recall	0.968	0.972	0.963
	F1-Score	0.978	0.983	0.969
Container Engine	Precision	0.968	0.991	0.984
	Recall	0.965	0.957	0.941
	F1-Score	0.966	0.974	0.962
Softirq Handling	Precision	0.982	0.984	0.962
	Recall	0.971	0.977	0.981
	F1-Score	0.976	0.980	0.971

and analyze the collective behavior of these attack groups, thus enhancing its capability to detect cgroups attacks.

E. Overhead (RQ3)

EsCapturer intercepts kernel events to obtain the execution information of containers, so it is necessary to measure the overhead introduced by EsCapturer. To this end, we leverage the FIO tool[19], which is a widely used I/O performance measurement tool, to analyze the performance difference between containers with and without EsCapturer. We focus on the I/O performance since EsCapturer intercepts critical I/O operations, which are performed with high frequency. We first measure the read/write performance without the behavior interceptor ten times, whose average values are 41,850 KB/s and 41,736 KB/s, respectively. We follow the same measurement method to test the performance with the interceptor, whose average values are 41,101 KB/s and 40,464 KB/s. Therefore, the average impact on performance is less than 1.7%. It shows that the overhead introduced by EsCapturer is low.

VII. DISCUSSION

The behavior interceptor of EsCapturer is designed based on the observation of cgroups attack vectors[2], so it may be

a limitation that the current attack vectors are not comprehensive. If there are newly discovered mechanisms that can be attacked to perform the cgroups escape attack, EsCapturer may not be able to obtain the corresponding behaviors. Fortunately, the proposed deep-learning-based anomaly detection approach of EsCapturer can model normal container behaviors and detect attacks that have different behavior intentions than normal behaviors. So, if a new attack vector is found, EsCapturer can detect the attacks by adding the corresponding behavior capture methods without changing the detection model.

VIII. RELATED WORK

This work aims to detect cgroups escape attacks for containers, so it is related with container security and anomaly detection.

A. Container Security

With the development of containers in cloud computing, container security [20, 21, 22, 23, 24] has become a hot topic. Containers use operating-system-level virtualization mechanisms and have their own application ecosystem, so new security risks are raised in container clouds.

Compared with virtual machines, containers share the host kernel, so their isolation is weaker than that of virtual machines[25, 26, 27, 28]. There are several isolation mechanisms to isolate and secure containers, such as namespaces, cgroups, Seccomp, etc. However, due to the complexity of the Linux kernels, it is difficult to isolate different containers on the same host completely[3, 29]. [3] has shown that some vulnerabilities (e.g., CVE-2017-7308[30], CVE-2017-5123[31], and CVE-2016-8655[32]) in syscalls can be exploited to perform container escape attacks. [2] breaks the limitation of cgroups in containers. Bastion [33] exploits the container network virtualization mechanism, which proves the weakness of container network isolation.

An important way to enhance container isolation is to run containers in light-weight virtual machines. The gVisor [34] leverages para-virtualization to isolate containers, which simulates most syscalls in the user space and redirects the network and file requests to the host OS. Unlike the gVisor, the Kata Containers [35] leverage hardware-assisted virtualization to isolate containers. In Kata Containers, every container runs

in a independent hardware-assisted virtual machine, so the isolation of containers is significantly enhanced.

Besides the VM-based containers, shrinking the attack surface of the host kernel is another way to secure containers. Syscalls are the biggest attack surface exposed to containers. Docker containers can access about 300 syscalls by default, so the vulnerabilities in syscalls can be exploited. To reduce the attack surface, [36, 37, 38, 39, 40] shrink the accessible syscalls for containers. [41] identifies the syscalls that are invoked by an application in different executing stages, then applies different Seccomp profiles in different stages to further shrink the attack surface. However, these approaches cannot solve the problem of cgroups escape attacks.

B. Behavior Analysis

After intercepting container behaviors, EsCapturer leverages deep learning technologies to analyze the behavior logs, such as system call sequences. Behavior-based anomaly detection approaches usually employ different kinds of learning models to identify abnormal behaviors.

Traditional machine learning-based methods have been widely used for system call sequence modeling and anomaly detection. Some techniques focus on feature-rich representations by extracting both fixed-length and variable-length subsequences via TF-IDF and suffix trees, and then apply a cost-sensitive relaxed SVM to handle class imbalance and noise [10]. Others adopt a more lightweight pipeline that converts system call sequences into TF-IDF weighted n-gram vectors, applies truncated SVD for dimensionality reduction, and uses decision tree classifiers to detect anomalies [11]. Another line of work models normal behavior as a centroid in a vector space, representing each input as a TF-IDF vector and identifying anomalies based on their distance from this centroid [12].

With the growing popularity of deep learning, models capable of capturing temporal dependencies have been introduced. For example, hybrid architectures integrating LSTM and Transformer modules aim to model both local and global behavior patterns from system call sequences [13]. CNN-based methods instead convert system call relationships into matrices and leverage convolutional layers to uncover spatial correlations relevant to malware activity [14]. More recently, a TCN-based method has been proposed to process system calls, binder calls, and composite behaviors in parallel, capturing long-range dependencies while maintaining high computational efficiency [15].

However, these approaches cannot extract correlated behaviors and learn behavior intentions based on comprehensive behavior parameter information, making it difficult to identify cgroups escape attacks where the attack behaviors are similar to normal behaviors.

IX. CONCLUSION

In this paper, a novel container behavior analysis approach based on behavior-intention learning, EsCapturer, is proposed to detect cgroups escape attacks. After analyzing the attack vectors, EsCapturer can capture the fine-grained attack-related

events in the host kernel with very low overhead. A multilevel deep learning model is proposed to learn the comprehensive local and contextual behavior intentions of related behavior sets. By analyzing the behavior intentions, the attack behaviors that are similar to normal behaviors can be detected precisely. The experimental results show that EsCapturer is able to detect container cgroups escape attacks accurately with very low overhead.

ACKNOWLEDGMENTS

This work was supported by the National Natural Science Foundation of China (Grants No. 62302122, 62172123), and the Heilongjiang Provincial Natural Science Foundation of China under Grants No. LH2023F017.

REFERENCES

- [1] J. Kosińska, B. Baliś, M. Konieczny, M. Malawski, and S. Zieliński, "Toward the observability of cloud-native applications: The overview of the state-of-the-art," *IEEE Access*, vol. 11, pp. 73 036–73 052, 2023.
- [2] X. Gao, Z. Gu, Z. Li, H. Jamjoom, and C. Wang, "Houdini's escape: Breaking the resource rein of linux control groups," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 1073–1086.
- [3] X. Lin, L. Lei, Y. Wang, J. Jing, K. Sun, and Q. Zhou, "A measurement study on linux container security: Attacks and countermeasures," in *Proceedings of the 34th Annual Computer Security Applications Conference*, 2018, pp. 418–429.
- [4] Y. Wang, Q. Wang, X. Chen, D. Chen, X. Fang, M. Yin, and N. Zhang, "Containerguard: A real-time attack detection system in container-based big data platform," *IEEE Transactions on Industrial Informatics*, 2020.
- [5] S. Locke, H. Li, T.-H. P. Chen, W. Shang, and W. Liu, "Logassist: Assisting log analysis through log summarization," *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3227–3241, 2021.
- [6] N. Sabharwal and P. Pandey, "Container application monitoring using sysdig," in *Monitoring Microservices and Containerized Applications*. Springer, 2020, pp. 235–269.
- [7] M. S. Haq, T. D. Nguyen, A. Ş. Tosun, F. Vollmer, T. Korkmaz, and A.-R. Sadeghi, "Sok: A comprehensive analysis and evaluation of docker container attack and defense mechanisms," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 4573–4590.
- [8] N. Alshuqayran, N. Ali, and R. Evans, "A systematic mapping study in microservice architecture," in *2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA)*. IEEE, 2016, pp. 44–51.
- [9] G. C. Fox, V. Ishakian, V. Muthusamy, and A. Slominski, "Status of serverless computing and function-as-a-service (faas) in industry and research," *arXiv preprint arXiv:1708.08028*, 2017.

- [10] X. Liao, C. Wang, and W. Chen, "Anomaly detection of system call sequence based on dynamic features and relaxed-svm," *Security and Communication Networks*, vol. 2022, no. 1, p. 6401316, 2022.
- [11] B. Subba and P. Gupta, "A tfidfvectorizer and singular value decomposition based host intrusion detection system framework for detecting anomalous system processes," *Computers & Security*, vol. 100, p. 102084, 2021.
- [12] N. Shamim, M. Asim, A. I. Awad, and M. K. Khan, "Anomaly detection in internet of things system calls using a centroid-based vector-space model," *IEEE Internet of Things Journal*, 2025.
- [13] Y. Guan and N. Ezzati-Jivan, "Malware system calls detection using hybrid system," in *2021 IEEE International Systems Conference (SysCon)*. IEEE, 2021, pp. 1–8.
- [14] A. Abderrahmane, G. Adnane, C. Yacine, and G. Khired-dine, "Android malware detection based on system calls analysis and cnn classification," in *2019 IEEE wireless communications and networking conference workshop (WCNCW)*. IEEE, 2019, pp. 1–6.
- [15] A. Joomye, M. H. Ling, M. B. Jasser, A. M. Ramly, and K.-L. A. Yau, "An effective temporal convolutional networks-based method for detecting android malware using dynamic extracted features," *IEEE Access*, 2025.
- [16] "Sysdig: Linux system exploration and troubleshooting tool with first class support for container," <https://github.com/draios/sysdig>.
- [17] "Trace-cmd: The front-end application to ftrace," <https://trace-cmd.org/>.
- [18] "ftrace: a look at ftrace," <https://lwn.net/Articles/322666/>.
- [19] "fio: Flexible i/o tester," <https://github.com/axboe/fio>.
- [20] S. Sultan, I. Ahmad, and T. Dimitriou, "Container security: Issues, challenges, and the road ahead," *IEEE Access*, vol. 7, pp. 52 976–52 996, 2019.
- [21] A. Tomar, D. Jeena, P. Mishra, and R. Bisht, "Docker security: A threat model, attack taxonomy and real-time attack scenario of dos," in *2020 10th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*. IEEE, 2020, pp. 150–155.
- [22] T. Bui, "Analysis of docker security," *arXiv preprint arXiv:1501.02967*, 2015.
- [23] D. Yu, Y. Jin, Y. Zhang, and X. Zheng, "A survey on security issues in services communication of microservices-enabled fog applications," *Concurrency and Computation: Practice and Experience*, vol. 31, no. 22, p. e4436, 2019.
- [24] J. Watada, A. Roy, R. Kadikar, H. Pham, and B. Xu, "Emerging trends, techniques and open issues of containerization: A review," *IEEE Access*, vol. 7, pp. 152 443–152 472, 2019.
- [25] T. Combe, A. Martin, and R. Di Pietro, "To docker or not to docker: A security perspective," *IEEE Cloud Computing*, vol. 3, no. 5, pp. 54–62, 2016.
- [26] X. Gao, Z. Gu, M. Kayaalp, D. Pendarakis, and H. Wang, "Containerleaks: Emerging security threats of information leakages in container clouds," in *2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2017, pp. 237–248.
- [27] X. Gao, B. Steenkamer, Z. Gu, M. Kayaalp, D. Pendarakis, and H. Wang, "A study on the security implications of information leakages in container clouds," *IEEE Transactions on Dependable and Secure Computing*, 2018.
- [28] A. A. Mohallel, J. M. Bass, and A. Dehghantaha, "Experimenting with docker: Linux container and base os attack surfaces," in *2016 International Conference on Information Society (i-Society)*. IEEE, 2016, pp. 17–21.
- [29] J.-A. Kabbe, "Security analysis of docker containers in a production environment," Master's thesis, NTNU, 2017.
- [30] "Cve-2017-7308," <http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-7308>.
- [31] "Cve-2017-5123," <http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-5123>.
- [32] "Cve-2016-8655," <http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2016-8655>.
- [33] J. Nam, S. Lee, H. Seo, P. Porras, V. Yegneswaran, and S. Shin, "Bastion: A security enforcement network stack for container networks," in *2020 USENIX Annual Technical Conference (USENIXATC 20)*, 2020, pp. 81–95.
- [34] "gvisor: A container sandbox runtime focused on security, efficiency, and ease of use." <https://gvisor.dev/>.
- [35] "Kata containers." <https://katacontainers.io/>.
- [36] L. Lei, J. Sun, K. Sun, C. Shenefiel, R. Ma, Y. Wang, and Q. Li, "Speaker: Split-phase execution of application containers," in *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 2017, pp. 230–251.
- [37] Z. Wan, D. Lo, X. Xia, L. Cai, and S. Li, "Mining sandboxes for linux containers," in *2017 IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2017, pp. 92–102.
- [38] S. Barlev, Z. Basil, S. Kohanim, R. Peleg, S. Regev, and A. Shulman-Peleg, "Secure yet usable: Protecting servers and linux containers," *IBM Journal of Research and Development*, vol. 60, no. 4, pp. 12–1, 2016.
- [39] S. Ghavamnia, T. Palit, A. Benameur, and M. Polychronakis, "Confine: Automated system call policy generation for container attack surface reduction," in *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*, 2020, pp. 443–458.
- [40] "Ibm. nabra containers: a new approach to container isolation." <https://nabra-containers.github.io/>.
- [41] S. Ghavamnia, T. Palit, S. Mishra, and M. Polychronakis, "Temporal system call specialization for attack surface reduction," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 1749–1766.